

Get the supporting sdk source code

If you feel that you don't know how to follow the document, you can directly view our pre-recorded hands-on instructional video.

<https://player.bilibili.com/player.html?aid=379890992&bvid=BV11Z4y1Q7pL&cid=462871023&page=1>

Configure the development environment

Our development environment is based on the ubuntu-18.04 system. In order to avoid wasting your time by climbing pits, please try to be consistent with us.

Install the lib & tool required for compilation

Next, install the corresponding compilation tools, etc.: You need to install some tools for SDK compilation, otherwise the compilation will fail.

```
book@100ask:~$ sudo apt-get install libc6-dev-i386
book@100ask:~$ sudo apt-get install lib32z1 lib32ncurses5
book@100ask:~$ sudo apt-get install libuuid1:i386
book@100ask:~$ sudo apt-get install cmake
book@100ask:~$ sudo apt-get install libncurses5-dev libncursesw5-dev
book@100ask:~$ sudo apt install bc
book@100ask:~$ sudo apt-get install xz-utils
book@100ask:~$ sudo apt-get install automake
book@100ask:~$ sudo apt-get install libtool
book@100ask:~$ sudo apt-get install libevdev-dev
book@100ask:~$ sudo apt-get install pkg-config
book@100ask:~$ sudo apt-get install openssh-server
book@100ask:~$ sudo apt-get install repo
```

Or use the following command to download the above tools all at once:

```
sudo apt-get install -y libc6-dev-i386 lib32z1 lib32ncurses5 libuuid1:i386
cmake libncurses5-dev libncursesw5-dev bc xz-utils automake libtool libevdev-
dev pkg-config openssh-server repo
```

It can be used after installation. The above tools && lib that need to be installed are required, because the relevant content will be used during the compilation process. I will not explain which error is one by one here. You can try to check the relevant errors without installing and compiling first. For convenience, you can install it at one time before compiling. If the default sh is not bash, you need to change sh to bash:

```
book@100ask:~$ sudo rm /bin/sh
book@100ask:~$ sudo ln -s /bin/bash /bin/sh
```

Get the source code

For the source code, we use the repo tool to manage multiple git repositories in a unified manner, which is convenient for one-click acquisition and update, and then use the following command to acquire the source code and wait for the acquisition to complete.

```
book@100ask:~$ git clone https://e.coding.net/codebug8/repo.git
book@100ask:~$ mkdir DongshanPiOne-TAKOYAKI && cd DongshanPiOne-TAKOYAKI
book@100ask:~/DongshanPiOne-TAKOYAKI$ ../repo/repo init -u
https://gitee.com/weidongshan/manifests.git -b linux-sdk -m
SSD202D/dongshanpi-one_takoyaki_dlc00v030.xml --no-repo-verify
book@100ask:~/DongshanPiOne-TAKOYAKI$ ../repo/repo sync -j4
```

The source code directory structure after successful acquisition is as follows

```
book@virtual-machine:~/DongshanPiOne-TAKOYAKI$ tree -L 1
.
├─ boot    //使用的是uboot-2015 主要用于烧写引导kernel启动。
├─ DevelopmentEnvConf  //自动配置开发环境的一些脚本文件
├─ gcc-arm-8.2-2018.08-x86_64-arm-linux-gnueabihf  //交叉编译工具链。
├─ kernel  //使用4.9.88版本的内核
├─ project //根文件系统和应用相关源码,主要包含了 编译构建根文件系统 APP 并自动打包生成合适的格式用来烧写。
└─ sdk
```

Configure the build environment

Set up the cross-compilation toolchain

The cross-compilation toolchain plays a very critical role in embedded Linux development and must be configured, otherwise subsequent operations cannot be performed.

Note : The `PATH=$PATH:/home/book` path specified here is the absolute path of the home directory of the demo host. You need to configure it according to the absolute path of your own home directory. The absolute path of the home directory can be obtained by using `cd ~` && `pwd` in the terminal.

- Permanent effect For permanent modification, please modify the user configuration file.
Note: If you do not use the vim command, you can use a graphical editing tool and execute: `gedit ~/.bashrc`

```
book@100ask:~$ vim ~/.bashrc
```

To add or modify at the end of the line, add the following lines:

```
export ARCH=arm
export CROSS_COMPILE=arm-linux-gnueabihf-
export PATH=$PATH:/home/book/DongshanPiOne-TAKOYAKI/gcc-arm-8.2-2018.08-x86_64-arm-linux-gnueabihf/bin
```

After setting, execute the `source ~/.bashrc` command to make it take effect. This command is to load the environment variables of these settings.

- Temporarily effective Note: Manually execute the "export" command to set the environment variable, this setting is only valid for the current terminal (you need to set it again when opening another terminal).

```
book@100ask:~$ export ARCH=arm
book@100ask:~$ export CROSS_COMPILE=arm-linux-gnueabihf-
book@100ask:~$ export PATH=$PATH:/home/book/DongshanPiOne-TAKOYAKI/gcc-arm-8.2-2018.08-x86_64-arm-linux-gnueabihf/bin
```

- Verify that the cross-compilation toolchain is normal by executing the `arm-linux-gnueabi-gcc -v` command in the terminal and outputting information similar to the following.

```
book@100ask:~$ arm-linux-gnueabi-gcc -v
Using built-in specs.
COLLECT_GCC=arm-linux-gnueabi-gcc
COLLECT_LTO_WRAPPER=/home/book/DongshanPiOne-TAKOYAKI/gcc-arm-8.2-2018.08-
x86_64-arm-linux-gnueabi/bin/./libexec/gcc/arm-linux-
gnueabi/8.2.1/lto-wrapper
Target: arm-linux-gnueabi
Configured with: /tmp/dgboter/bbs/bc-b1-2-11--
rhe6x86_64/buildbot/rhe6x86_64--arm-linux-gnueabi/build/src/gcc/configure
--target=arm-linux-gnueabi --prefix= --with-sysroot=/arm-linux-
gnueabi/libc --with-build-sysroot=/tmp/dgboter/bbs/bc-b1-2-11--
rhe6x86_64/buildbot/rhe6x86_64--arm-linux-gnueabi/build/build-arm-linux-
gnueabi/install//arm-linux-gnueabi/libc --enable-gnu-indirect-function
--enable-shared --disable-libssp --disable-libmudflap --disable-
libsanitizer --enable-checking=release --enable-languages=c,c++,fortran --
with-gmp=/tmp/dgboter/bbs/bc-b1-2-11--rhe6x86_64/buildbot/rhe6x86_64--arm-
linux-gnueabi/build/build-arm-linux-gnueabi/host-tools --with-
mpfr=/tmp/dgboter/bbs/bc-b1-2-11--rhe6x86_64/buildbot/rhe6x86_64--arm-
linux-gnueabi/build/build-arm-linux-gnueabi/host-tools --with-
mpc=/tmp/dgboter/bbs/bc-b1-2-11--rhe6x86_64/buildbot/rhe6x86_64--arm-linux-
gnueabi/build/build-arm-linux-gnueabi/host-tools --with-
isl=/tmp/dgboter/bbs/bc-b1-2-11--rhe6x86_64/buildbot/rhe6x86_64--arm-linux-
gnueabi/build/build-arm-linux-gnueabi/host-tools --with-arch=armv7-a --
with-fpu=neon --with-float=hard --with-arch=armv7-a --with-pkgversion='GNU
Toolchain for the A-profile Architecture 8.2-2018-08 (arm-rel-8.23)'
Thread model: posix
gcc version 8.2.1 20180802 (GNU Toolchain for the A-profile Architecture
8.2-2018-08 (arm-rel-8.23))
book@100ask:~$
```

After setting the cross-compilation tool and linking it down, you can start compiling and programming.